I. Đánh giá các thuật toán indexing

Bảng kết quả (Dataset: 10000 dòng)

Index	Build Time (s)	Search Time 100q (ms)	Size (MB)	Recall@10	Recall@50	Recall@100
Flat	0.0085	18.56	14.65	1.0000	1.0000	1.0000
HNSW	0.2389	2.11	17.24	0.9990	0.9708	0.8529
IVF	0.1303	3.12	14.87	0.9970	0.9736	0.9370
IVF+PQ	1.2496	13.16	0.68	0.6390	0.7066	0.7036

Bảng kết quả (Dataset: 50000 dòng)

Index	Build Time (s)	Search Time 100q (ms)	Size (MB)	Recall@10	Recall@50	Recall@100
Flat	0.0508	60.24	73.24	1.0000	1.0000	1.0000
HNSW	2.0466	2.60	86.22	0.9880	0.9464	0.8174
IVF	2.1703	5.07	74.21	0.9800	0.9444	0.9095
IVF+PQ	7.7560	8.40	1.73	0.6110	0.5866	0.5711

Diễn giải & Phân tích tổng quát

Dựa vào các kết quả trên, ta có thể rút ra một số đặc tính nổi bật của từng loại Index:

1. **Flat (IndexFlatIP):**
   - **Tốc độ Search:** Rất chậm khi dữ liệu lớn (tìm kiếm quét toàn bộ - exhaustive search). Thời gian tìm kiếm tăng tuyến tính thuận theo số dòng.
   - **Độ chính xác (Recall):** Đạt 1.0 (hoàn hảo) do không có phép xấp xỉ nào. Các index khác đều dùng Flat làm mốc so sánh (Ground Truth).

- **Memory (Size):** Chiếm dung lượng ở mức cơ bản bằng chính ma trận dữ liệu, không sinh ra thêm bộ nhớ phụ lớn như HNSW.

2. **HNSW (Hierarchical Navigable Small World):**
   - **Tốc độ Search:** Cực kỳ nhanh nhất trong tất cả các thuật toán. Việc tìm đường trên đồ thị nhiều tầng giúp độ phức tạp đạt mức log(N).
   - **Độ chính xác (Recall):** Thường rất sát mức 1.0 (trên 99%).
   - **Memory (Size):** Đây là điểm yếu của HNSW. Index phình to một cách đáng kể (nhiều MB hơn Flat) vì phải lưu trữ cả không gian vector lẫn ma trận đỉnh/cạnh của đồ thị đa tầng.
   - **Build Time:** Rất lâu. Việc xây dựng và kết nối các nút trên đồ thị yêu cầu nhiều tính toán.
3. **IVF (Inverted File Index):**
   - **Tốc độ Search:** Nhanh hơn đáng kể so với Flat vì không phải duyệt toàn bộ, mà chỉ duyệt ở các nhóm (clusters) gần nhất với query (`nprobe`).
   - **Build Time:** Nhanh hơn HNSW nhưng chậm hơn Flat (do Flat gần như Build = 0). Nó tốn một khoảng thời gian nhỏ chạy K-Means clustering trong hàm `train` .
   - **Memory (Size):** Rất nhỏ gọn. Không lưu đồ thị như HNSW, chỉ lưu centroids và danh sách vector.
   - **Độ chính xác (Recall):** Rất tốt (mặc định với nprobe đủ lớn), nhưng có thể bị rớt (mis-hits) các vector nằm ngay rìa ranh giới giữa 2 clusters.
4. **IVF+PQ (Product Quantization):**
   - **Memory (Size):** Điểm mạnh tuyệt đối. Thuật toán phân rã và nén vector này ép kích thước bộ nhớ (Size MB) xuống chỉ bằng 1/10 đến 1/20 so với thuật toán Flat. Rất thích hợp để triển khai trên các cụm máy chủ thiếu RAM hoặc cho hàng tỷ vector.
   - **Độ chính xác (Recall):** Thường là thuật toán có tỷ lệ Recall thấp nhất. Việc lượng hóa (quantization) làm xấp xỉ vector dẫn tới tính khoảng cách bị sai số. Do đó độ chính xác tìm điểm gần nhất kém đi.
   - **Tốc độ Search:** Rất nhanh (chỉ thao tác trên bảng look-up table).

II. Tuning tham số

Dataset: 50,000 vectors | Số lượng Queries: 1,000

1. Tuning HNSW (`efSearch`)

efSearch	Search Time 1000q (ms)	Recall@10	Recall@50	Recall@100
16	30.55	0.9927	0.9531	0.8232

efSearch	Search Time 1000q (ms)	Recall@10	Recall@50	Recall@100
32	34.88	0.9946	0.9724	0.8913
64	56.62	0.9981	0.9882	0.9535
128	126.04	0.9996	0.9959	0.9849
256	177.79	0.9996	0.9984	0.9948

2. Tuning IVF (`nprobe`)

nlist = 400

nprobe	Search Time 1000q (ms)	Recall@10	Recall@50	Recall@100
1	10.09	0.8390	0.6994	0.5608
5	26.20	0.9634	0.9028	0.8335
10	45.88	0.9814	0.9435	0.8992
20	82.81	0.9903	0.9700	0.9450
50	197.57	0.9970	0.9881	0.9787
100	398.24	0.9993	0.9963	0.9929

3. Tuning IVF+PQ (`nprobe`)

nlist = 400, m = 8

nprobe	Search Time 1000q (ms)	Recall@10	Recall@50	Recall@100
1	19.41	0.5694	0.5567	0.5042
5	57.14	0.6013	0.6014	0.5722
10	101.62	0.6026	0.6044	0.5773
20	154.76	0.6037	0.6058	0.5791
50	351.10	0.6038	0.6062	0.5795

nprobe	Search Time 1000q (ms)	Recall@10	Recall@50	Recall@100
100	717.19	0.6038	0.6063	0.5797

Diễn giải sự đánh đổi (Trade-off) giữa độ chính xác và tốc độ

`efSearch` (HNSW)

- **Cơ chế:** Tham số `efSearch` quyết định thuật toán sẽ giữ bao nhiêu ứng viên trong hàng đợi (queue) ở quá trình duyệt đồ thị để tìm đường đến điểm gần nhất.
- **Trade-off:** Việc tăng `efSearch` sẽ giúp Recall tăng lên tiệm cận 1.0 (vì khả năng bỏ sót đường đi ngắn nhất giảm đi), nhưng bù lại làm cho **Search Time tăng cao hơn** do phải tính toán và duy trì nhiều nút hơn trong hàng đợi.

`nprobe` (IVF và IVF+PQ)

- **Cơ chế:** Thuật toán IVF (K-Means) đã chia 50,000 vectors vào `nlist = 400` cụm. Khi có query tới, nó sẽ xác định cụm nào có trung tâm (centroid) gần với query nhất và quét bên trong cụm đó. Tham số `nprobe` quyết định hệ thống sẽ quét bao nhiêu cụm gần nhất.
- **Trade-off:**
  - Khi `nprobe = 1` , tốc độ cực kì nhanh (vì chỉ quét 1/400 tổng số vectors), nhưng Recall rất thấp vì có khả năng điểm gần nhất thực sự lại nằm ở rìa cụm lân cận.
  - Khi `nprobe` tăng dần (ví dụ lên 50 hay 100), Recall tăng mạnh, nhưng **Search Time cũng tăng** vì số vector phải duyệt tăng tỷ lệ thuận với số cụm. (Nếu nprobe = 400, nó trở về tương đương với vét cạn Flat).
- **IVF vs IVF+PQ:** Ở bảng IVF, khi nprobe đủ lớn, Recall có thể đạt tới 1.0. Nhưng với **IVF+PQ**, vì các vector bên trong cụm đã bị nén (quantization) làm giảm độ chính xác, dù bạn có tăng nprobe lên tối đa thì Recall cũng sẽ chững lại và **không thể đạt 1.0**. Sự hi sinh độ chính xác này là bắt buộc để đổi lấy lợi thế cực lớn về Memory (Size).